

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico parte 2

Programación Dinámica

[illegible]

3 de diciembre de 2024

Nombre	Padrón
Maximo Giovanettoni	110303
Franco Bossi	111122
Luca Saluzzi	108088
Agustin Conti	107800
Gabriel Peralta	101767

1. Introducción

En este trabajo se aborda la resolución de un juego de mesa de 2 jugadores en donde, dado un arreglo de monedas, cada jugador extrae una moneda de uno de los extremos. El que consigue el valor máximo acumulado es el ganador. Para buscar cuál es la jugada óptima se utiliza la técnica de Programación Dinámica.

En el siguiente informe se detalla el algoritmos planteado, la ecuación de recurrencia, y una demostración de que la ecuación permite obtener el máximo valor posible. Por último se hace un análisis de la complejidad temporal, y mediciones que permitan corroborar la complejidad teórica.

Resolución del problema

2. Recurrencia del problema

La idea básica es que dos jugadores están tomando monedas de una secuencia de monedas. Cada jugador quiere maximizar la cantidad de monedas que puede tomar. Dado que cada jugador puede elegir la primera o la última moneda de la secuencia en su turno, el problema es inherentemente recursivo.

La ecuación de recurrencia se define de la siguiente manera:

■ $OPT[izq][der]$

representa el valor máximo que puede obtener el jugador que toma monedas entre las posiciones izq y der .

■ El jugador tiene dos opciones:

2.1. Tomar la moneda izquierda:

En este caso, el jugador toma $monedas[izq]$ y el siguiente jugador puede tomar monedas entre las posiciones $izq + 1$ y der . El valor acumulado será:

```
1 tomar_izq = monedas[izq] + min(OPT[izq + 2][der], OPT[izq + 1][der - 1])
```

Aquí, el jugador siguiente juega de forma óptima, lo que significa que tratará de minimizar lo que el jugador original puede obtener en su siguiente turno.

2.2. Tomar la moneda derecha:

En este caso, el jugador toma $monedas[der]$ y el siguiente jugador toma monedas entre las posiciones izq y $der - 1$. El valor acumulado será:

```
1 tomar_der = monedas[der] + min(OPT[izq + 1][der - 1], OPT[izq][der - 2])
```

Finalmente, el jugador tomará la opción que le ofrezca el valor máximo:

```
1 OPT[izq][der] = max(toma_izq, tomar_der)
```

2.3. Demostración inductiva

Usaremos inducción para demostrar que la solución dada por la ecuación de recurrencia es correcta y maximiza el valor acumulado.

2.4. Base inductiva:

Para un solo elemento, $OPT[i][i] = monedas[i]$. Esto es correcto, ya que si solo queda una moneda, el jugador necesariamente la tomará, y ese será su valor máximo.

2.5. Paso inductivo:

Supongamos que la recurrencia es correcta para cualquier subsecuencia de longitud menor o igual a $k-1$. Ahora demostraremos que también es válida para una subsecuencia de longitud k .

El jugador tiene dos opciones, tomar la moneda izquierda o derecha. Si toma la izquierda, el valor que obtiene es el valor de esa moneda más lo que puede maximizar en las monedas restantes,

sabiendo que el siguiente jugador tomará de forma óptima. El mismo razonamiento aplica si toma la moneda derecha.

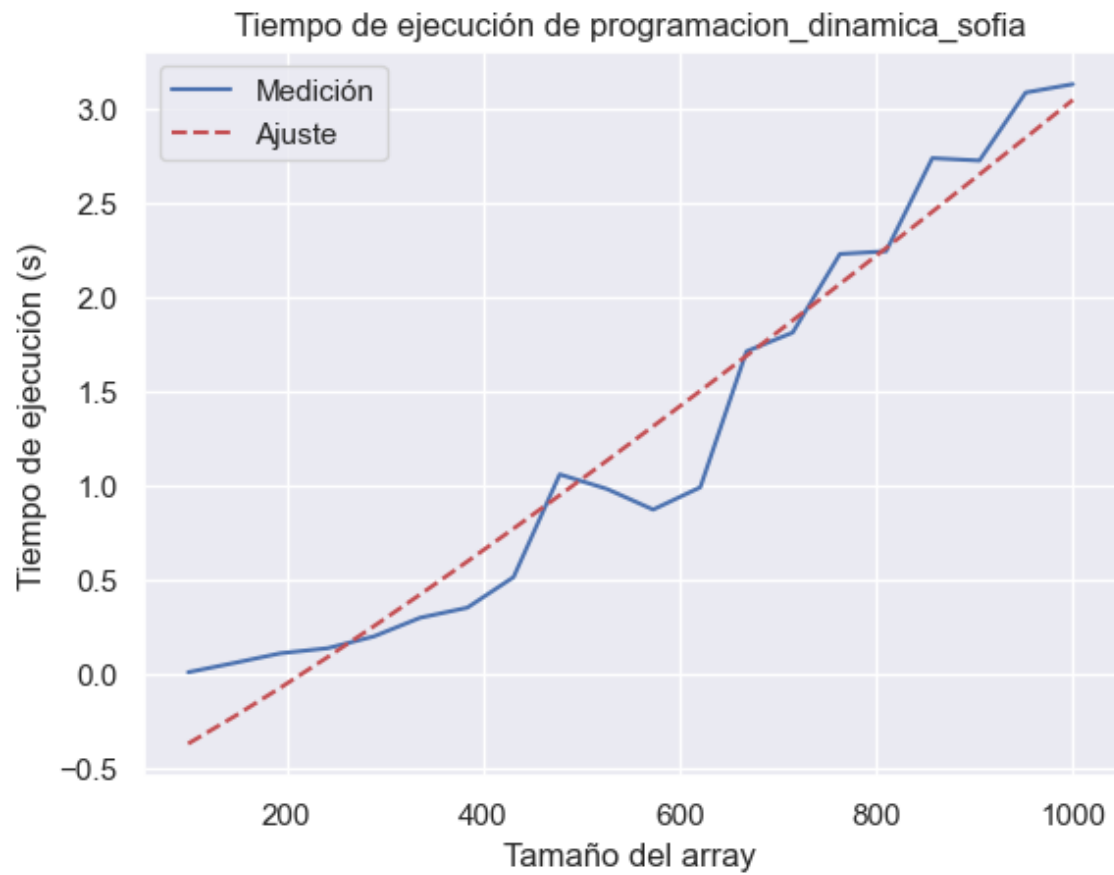
El hecho de que tomemos el mínimo de las opciones del jugador siguiente asegura que estamos considerando que ambos jugadores juegan de forma óptima, lo que maximiza el valor acumulado para el primer jugador.

Dado que la recurrencia es válida para subsecuencias más pequeñas y seguimos esta lógica inductiva para subsecuencias más grandes, hemos demostrado por inducción que la recurrencia lleva al valor máximo acumulado.

3. Mediciones

Se realizaron mediciones en base a crear arreglos de diferentes largos, yendo de 100 en 100 elementos, donde los elementos en cada caso fueron generados por los valores pseudoaleatorios del lenguaje (el módulo `random`).







Como se puede apreciar, ambos algoritmos tienen una tendencia efectivamente lineal en función del tamaño de la entrada, si bien el algoritmo iterativo es más veloz en cuestión de constantes.

4. Conclusiones

La conclusion que se pudo sacar en este informe es que, aunque el algoritmo disponga de una gran simplicidad y su entendimiento en sí es más fácil que otros algoritmos, no siempre se llega al óptimo (por el hecho de que hay casos los cuales se hace imposible a Sofia a ganar), la complejidad no es muy conveniente en casos los cuales hay mucho volumen en el problema. Aún así, existen muchos casos los cuales Sofia gana y el algoritmo cumple un rol significativo en ese proceso por el hecho de que le calcula el óptimo en cada caso y eso es clave en muchos casos (en muchos los cuales estuvieron plasmandos en los tests).Entonces, el algoritmo es conveniente para ser usado en muchos casos pero no puede ser considerado como el mejor algortimo posible para resolver el problema.